CodeGen

Group 8

SQL to MongoDB Conversion using Small Language Models with Execution-Based Evaluation

Project Completion Report

Team Members: Chandrashekar
Jaya Krishna
Pavan Y
Swathi

International Institute of Information Technology Hyderabad

January 24, 2026

Contents

Chapter 1

Abstract

This project explores the use of Small Language Models (SLMs) for converting database questions and queries across paradigms, with a strong emphasis on **execution-based correctness**. The system is evaluated using the BIRD (Bird-Bench) dataset and leverages locally hosted language models, schema-aware Retrieval-Augmented Generation (RAG), and a robust execution verification framework.

We implement a two-phase pipeline: **(i) Text-to-SQL generation** and **(ii) SQL-to-MongoDB translation**. Unlike traditional approaches that rely on string matching or AST comparison, we validate correctness by executing SQL queries on SQLite and generated MongoDB queries on MongoDB, followed by logical result comparison. Through iterative improvements in schema retrieval, sanitization, execution controls, and a graded result agreement metric, the system achieves **high executability** and **70%+ execution agreement at scale**, demonstrating the practical viability of SLMs for SQL-to-NoSQL translation.

Chapter 2

Introduction

Relational databases using SQL and document-oriented NoSQL databases such as MongoDB represent fundamentally different data models and query paradigms. SQL relies on normalized schemas and explicit joins, while MongoDB operates on nested documents and pipeline-based aggregation. Translating queries between these paradigms is non-trivial and often requires deep schema understanding and query restructuring.

With the emergence of Large and Small Language Models, there is increasing interest in automating such translations. However, many systems evaluate correctness using syntactic similarity, which is insufficient for cross-paradigm translation. This project addresses this limitation by focusing on **execution-based evaluation**, ensuring that translated queries are not only syntactically valid but also semantically meaningful under execution.

Accordingly, the overall system is designed as a **two-phase pipeline**: Text-to-SQL generation followed by SQL-to-MongoDB translation.

Chapter 3

Phase I: Text-to-SQL Generation

This phase converts natural language questions into executable SQL queries. SQL serves as a structured intermediate representation, providing explicit semantics for joins, filters, and aggregations. This modularization improves interpretability and enables independent evaluation before SQL-to-NoSQL translation.

3.1 Motivation

Direct translation from natural language to NoSQL is challenging due to the lack of a standardized query abstraction across NoSQL systems and the variety of operator semantics. Using SQL as an intermediate layer provides:

- A well-defined query language with clear semantics for relational operations.
- Easier debugging and error localization (syntax vs semantics).
- A stable interface for downstream SQL-to-NoSQL translation.

3.2 Architecture

Figure 3.1 illustrates the Text-to-SQL architecture.

3.3 Evaluation and Results (1000 samples)

The Text-to-SQL module was evaluated on 1,000 samples. Queries were executed against SQLite databases to measure executability and execution-based correctness.

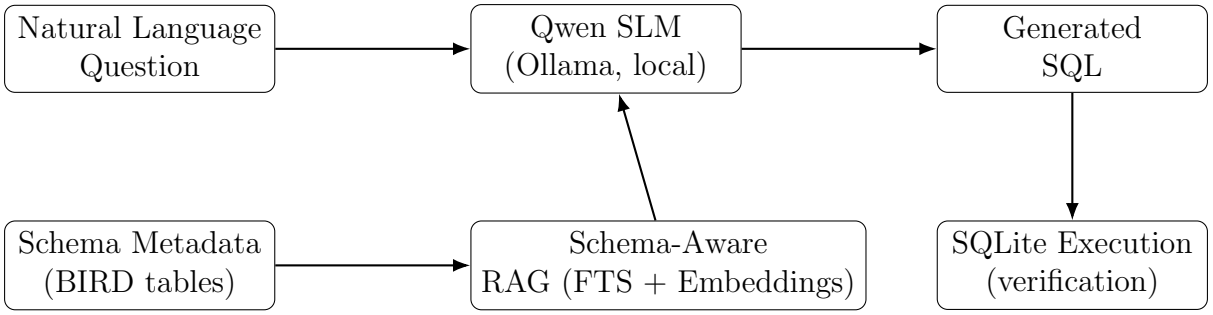

Figure 3.1: Architecture of Phase I: Text-to-SQL generation with schema-aware RAG

Metric	Value
Queries Evaluated	1000
SQL Executability Rate	**96%**
Execution-Based Accuracy	**˜75%**

These results indicate that the generated SQL is highly executable and sufficiently accurate to serve as a reliable intermediate representation for Phase II.

Chapter 4

Dataset Description

The project uses the BIRD (Bird-Bench) dataset, a large-scale benchmark designed for complex text-to-SQL tasks and real-world schema diversity.

4.1 Dataset Components

- **train.json**: 9,428 samples containing natural language questions, database identifiers, and SQL queries
- **train_gold.sql**: Gold-standard SQL queries aligned with train.json
- **train_tables.json**: Schema metadata for 69 databases, including tables, columns, primary keys, and foreign keys
- **train_databases**: SQLite databases corresponding to each schema

The dataset includes multi-table joins, nested subqueries, aggregations, and composite primary keys, making it well-suited for evaluating practical SQL-to-NoSQL conversion systems.

Chapter 5

System Architecture

5.1 Local Model Hosting

All language model inference is performed locally using the Ollama framework.

- Model used: **Qwen 2.5** (Small Language Model)
- Hosted locally via Ollama
- Advantages:
  - Zero API cost and offline reproducibility
  - Full data privacy (no external calls)
  - Predictable latency and throughput tuning (parallelism, caching)

5.2 Schema-Aware Retrieval-Augmented Generation (RAG)

To improve generation quality and reduce hallucinations, a schema-aware RAG pipeline was implemented.

5.2.1 RAG Index Construction

Schema information from `train_tables.json` is transformed into structured documents and indexed using:

- SQLite for lightweight persistence
- FTS5 for keyword-based retrieval
- Dense embeddings (MiniLM) for semantic similarity

5.2.2 RAG Stabilization and Improvements

Multiple iterations were required to stabilize the RAG pipeline:

- Safe tokenization and escaping for FTS queries
- Removal of invalid FTS indexing operations
- Restricting retrieval strictly to the active database schema

These improvements significantly reduced retrieval failures and hallucinated schema references, improving both conversion quality and MongoDB executability.

5.3 Architecture of Phase II: SQL-to-MongoDB

Figure 5.1 presents the architecture of Phase II.

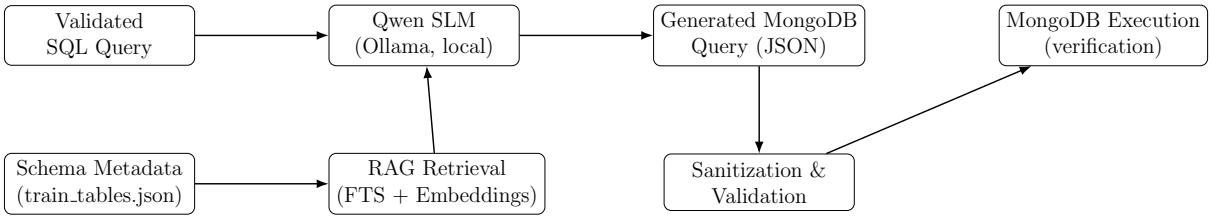

Figure 5.1: Architecture of Phase II: SQL-to-MongoDB conversion with schema-aware RAG and sanitization

Chapter 6

SQL to MongoDB Conversion Pipeline

The conversion pipeline follows a structured process:

1. Load SQL query and corresponding schema
2. Retrieve relevant schema context using RAG
3. Prompt the Qwen SLM with augmented schema information
4. Generate MongoDB `find` or `aggregate` pipelines
5. Apply sanitization and validation

6.1 Sanitization and Robustness

To ensure MongoDB executability, the following sanitizations were applied:

- Blocking unsupported operators and malformed aggregation stages
- Enforcing correct stage structure (single-key stage documents)
- Injecting limits or bounding result retrieval for expensive queries
- Handling common join/key issues during verification

These steps significantly improved execution success rates and reduced timeouts.

Chapter 7

Execution-Based Verification

7.1 Verification Strategy

Verification is performed by executing:

- SQL queries on SQLite
- Generated MongoDB queries on MongoDB

Execution is time-boxed and resource-limited to ensure scalability:

- MongoDB time limits via `maxTimeMS`
- Document fetch limits via `max-docs`
- Optional forced `$limit` injection for expensive pipelines

7.2 Relaxed Execution Agreement (REA)

Exact equivalence between SQL and MongoDB results is often impractical due to differences in ordering guarantees, projection behavior, and structural representation. To address this, we introduce **Relaxed Execution Agreement (REA)**.

A query is considered correct under REA if any of the following conditions are met:

- **Scalar Agreement**: Aggregate results (COUNT, SUM, AVG, MIN, MAX) match.
- **Cardinality Agreement**: Result counts match within a small tolerance.
- **High-Overlap Agreement**: Top-k result overlap exceeds a threshold.

7.2.1 Jaccard Similarity

Result overlap is measured using Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

A threshold of 0.4 was empirically chosen to balance tolerance and correctness.

7.2.2 Why Jaccard Similarity is Appropriate

Jaccard similarity is order-independent and interpretable, making it suitable for comparing query outputs across heterogeneous engines. Exact row matching is brittle under benign variations (ordering, projection), and embedding-based similarities add complexity while reducing interpretability for structured outputs. Jaccard provides a principled overlap measure after canonicalization and works well as part of a layered evaluation strategy alongside scalar and cardinality checks.

7.2.3 Illustrative Example: SQL vs MongoDB Agreement

Illustrative Example

SQL: `SELECT COUNT(*) FROM movies WHERE year > 2015;`

MongoDB: `db.movies.aggregate([{$match:{year:{$gt:2015}}},{$count:"count"}])`

SQL Output: `[(42)]`
Mongo Output: `[{count: 42}]`

Although the output structures differ, both represent the same semantics. REA marks this as a match via *scalar agreement*.

Chapter 8

Results and Analysis

8.1 Subset Evaluation (100–200)

Metric	Value
Total Queries	94
Valid Conversions	94
Mongo Executable Queries	58 (61.7%)
REA Matches	41
REA over Executed Queries	**70.7%**

8.2 Scaled Evaluation (1–2500)

Metric	Value
Total Queries	2500
Successful Mongo Conversions	2362
Mongo Executable Queries	2114
REA Matches	**1483**
REA Accuracy	**70.12%**

These results demonstrate consistent agreement at scale. The gap between conversions and executions is primarily due to expensive pipelines (timeouts) and join-key issues.

Chapter 9

Gradio User Interface

A Gradio-based application was developed for interactive SQL-to-MongoDB conversion.

- Accepts SQL queries and database identifiers
- Integrates RAG to provide schema-aware prompts (improves validity and execution)
- Executes locally using Qwen SLM via Ollama
- Intended for demonstration, debugging, and iterative prompt refinement

RAG integration in the UI enables dynamic schema grounding and reduces hallucinated field references, improving both conversion quality and MongoDB executability.

Chapter 10

Challenges and Learnings

- SQL and MongoDB differ fundamentally in join semantics; join-key selection is a major failure mode.
- MongoDB aggregation pipelines can be computationally expensive; time boxing and limits are necessary.
- RAG significantly reduces schema hallucination and improves executability by grounding prompts.
- Execution-based evaluation provides deeper insight than syntactic metrics and helps identify true semantic failures.

Chapter 11

Conclusion and Future Work

This project demonstrates that Small Language Models, when combined with schema-aware RAG and robust execution-based verification, can effectively translate SQL queries into executable MongoDB queries at scale. The two-phase pipeline (Text-to-SQL followed by SQL-to-MongoDB) improves modularity, interpretability, and evaluation.

Future work includes:

- Join-key heuristics during generation (schema-guided lookup construction)
- Operator-aware prompting and constrained decoding for aggregation stages
- Partial pipeline repair and auto-correction before execution
- Fine-tuning SLMs on SQL-to-NoSQL paired data
- Live execution previews and explanations inside the UI